marp: true
theme: default
paginate: true
size: 16:9
style: |
section { font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif; }
h1 { color: #1a1a2e; }
h2 { color: #16213e; }
table { font-size: 0.85em; }
code { background: #f5f5f7; padding: 1px 4px; border-radius: 3px; }
blockquote { border-left: 4px solid #16213e; padding-left: 12px; color: #444; }

The problem

A trader, risk officer, or surveillance analyst needs to know **when something unusual is happening, in the moment.**

Existing tools are either:

- **Too slow** (EOD batch reports, alerts come hours after the move)
- **Too noisy** (every fixed-threshold breach fires; staff learn to ignore them)
- **Too narrow** (single-asset z-scores miss systemic / cross-asset events)

Qpulse: streaming statistics tuned for low false-positive rate, with two complementary detection tiers running side-by-side.

Two-tier architecture

```
         Live WS feed (Alpaca, etc.)
                   │
                   ▼
    ┌─────────────────────────────────┐
    │  HOT PATH — sub-millisecond     │   per-asset detectors
    │  cusum / robust_z / regime      │   on price + microstructure
    │  spread / OBI / quote_rate      │
    │  + burst meta-detector          │
    └─────────────────────────────────┘
                   │
                   ▼  same alert bus
    ┌─────────────────────────────────┐
    │  BATCH PATH — daily refit        │   t-copula on multi-asset
    │  dependence_shift                │   correlation breaks
    └─────────────────────────────────┘
                   │
         WebSocket UI · SQLite · webhook
```

Hot path catches single-asset shocks. **Batch path** catches systemic/correlation

What you see when it runs

- Live tape, color-coded by alert kind
- Throughput / p50 / p95 / p99 latency
- Alert density per symbol (last 5 min)
- Click any alert → price chart with marker
- Pause / resume / per-symbol threshold tuning

Open

screenshot placeholder

The alert palette

Kind	Detects	Data
`robust_z`	Single-tick price outlier	trades / mid
`cusum`	Cumulative price drift	trades / mid
`regime_shift`	Distribution of returns changed	trades / mid
`spread_widen`	Bid-ask spread vs baseline	quotes
`obi_extreme`	Order book one-sided wall	quotes
`quote_rate_spike`	Quote arrival rate jump	quotes
`burst`	Alerts clustering in time	other alerts
`dependence_shift`	Cross-asset correlation break	daily multi-asset

Each kind targets a distinct failure mode of "normal" market behavior.

Algorithms — hot path

Streaming statistics — O(1) memory, O(1) per tick:

- **EWMA mean + MAD** for fast-moving baselines
- **P² quantile estimator** (Jain & Chlamtac, 1985) for spread / quote-rate where score-magnitude matters — avoids the EWMA-MAD self-inflation ceiling at $z \approx 39.89$
- **Two-sided CUSUM** (Page, 1954) for cumulative drift
- **Two-sample KS test** every 50 ticks: short window vs subsampled reference → distributional regime change
- **OBI hysteresis**: fire at $|OBI| > 0.98$, re-arm at $|OBI| < 0.5$ — prevents flip-flop on naturally imbalanced books
- **Burst (Hawkes-degenerate)**: rolling alert count vs EWMA baseline — captures cross-kind clustering

Algorithms — batch path

t-copula on rank-transformed multi-asset returns:

```
Each column → empirical CDF → uniform marginals
Correlation R via Kendall τ:  R_ij = sin(π · τ_ij / 2)
Degrees of freedom: max-LL grid search over {3, 5, 10, 20}
Score: rolling 5-day log-likelihood vs 248-point null distribution
Fire when |z| ≥ threshold
```

Detects when the *joint dependence structure* of an asset basket departs from its reference period. Catches events the hot path is structurally blind to: SPY can be flat while bond-equity-credit correlation rotates 90°.

Why t-copula and not MPS-copula? → next slide.

The MPS-copula gate

We considered MPS-copula (matrix product state, "quantum-inspired"). Pre-registered 5 pass criteria, 3 crisis events (COVID, Terra/Luna, SVB), 3 negative-control quiet windows.

Criterion	Result
MPS detection ≥ 1.5× Gaussian on ≥2 events	❌ 1/3
MPS timing ≤ Gaussian + 2d	❌ 0/2
MPS > t-copula on ≥1 event	✅ 1/3
MPS spurious ≤ 1 of 3 quiet windows	✅
MPS not more spurious than t	❌

MPS failed 3 of 5 criteria. We shipped t-copula.

Validation — hot path on BTC

10 canonical crypto crisis events + 6 explicitly-quiet 7-day windows.

Metric	Value
Recall	**60%** (6/10)
Median time-to-detect	**0 ms**
False positive rate	**0.05/day** in quiet markets

Caught: Terra/Luna (2022-05-09/10), FTX bank run + Chapter 11, USDC depeg, SVB contagion.
Missed: events outside data window, gradual moves (Celsius mid-decline, China ban early in calibration).

Validation — batch path on macro basket

8-asset core: 4 sectors + SPY + TLT + GLD + HYG. 9 macro events + 5 quiet windows.

Metric	Value
Recall	**67%** (6/9)
False positive rate	**0.00/day** in quiet markets

Caught: COVID Black Monday, Fed emergency cut, Fed unlimited QE, CPI shock 2022, SVB failure, Signature Bank.

Three of these were invisible to the hot path — SPY didn't move dramatically, but cross-asset correlation broke. That's the unique value the batch tier provides.

Performance

Benchmark	Value
End-to-end p99 latency (real Alpaca WS)	< 500 µs
Throughput (single core, synthetic)	280k ticks/sec
Memory per per-symbol detector	O(1) streaming
Backtest throughput	130–250k ticks/sec

Pure Python (numpy, scipy, fastapi, uvloop, websockets, pandas, yfinance). No proprietary deps. No quantum SDK. Single-process FastAPI + asyncio. Runs on a laptop or a t3.medium.

What's already in the box

- Live WebSocket UI with click-through tape, sparklines, alert density bars
- Auto `.env` loading, optional Bearer-token auth, health endpoint with feed liveness
- SQLite persistence (WAL mode, batched writes)
- Webhook sink (zero new deps, exponential backoff)
- Replay tool (CSV → live), recorder (live → CSV), backtester, parameter sweep, latency benchmark
- Pre-registered MPS gate + reproducible verdict
- Labeled-event evaluator with positive + negative-control support
- 7 alert kinds in hot path + 1 in batch path

All running today. Validation numbers are reproducible from the repo.

Roadmap

Phase	Status
0–2: foundation, multi-alert, productionization	✅ done
0.5: labeled-event evaluator + benchmarks	✅ done
1: regime + burst detectors	✅ done
3.5: MPS gate (decided to ship t-copula)	✅ done
4: cross-asset batch path	✅ done
5: Qmetrum integration via Qlens (forecasting + LLM agent)	☐ next
6: dashboard polish, public demo URL, backfill	☐
Catalog expansion (broader event labels)	☐ ongoing

Why this is a real product

1. **Streaming math, not ML.** No model retraining, no GPUs, no drift. Works from cold start, adapts via EWMA/P^2, stays calibrated.
2. **Honest validation.** Pre-registered criteria, negative controls, public decision trail. Not "we ran it on a few days that looked good."
3. **Sub-millisecond hot path.** Real measurement, not aspirational. Headroom for richer feature engineering before latency becomes a constraint.
4. **Two complementary tiers.** They don't compete — they cover different failure modes (asset shock vs. systemic correlation break).
5. **Operationally clean.** Persistence, auth, health, webhooks, replay, backtest, evaluator — all already there, no future work to "make it productionizable."

Q & A — collaboration angles

If you're interested in working on it, areas with the most leverage:

- **Catalog expansion.** More labeled events → tighter validation → better marketing claims.
- **Qmetrum / Qlens integration.** Tie real-time alerts to LLM-narrated context from a forecasting platform.
- **Microstructure depth.** Order-book depth metrics beyond top-of-book, OFI signals, microprice variants.
- **Alpaca SIP / Polygon paid tiers.** Move from IEX-only crypto to full US equities tick.
- **Distribution.** Pricing model, target customers (prop trading shops, risk desks, exchanges, surveillance vendors), go-to-market.

The system is real and validated. The remaining work is product/distribution, not "make

Demo

`python -m app.live` → `http://localhost:8080/`

`python -m app.inspect_alerts` → live alert summary

`python -m app.backtest --input data.csv` → run on any historical CSV

`python -m app.evaluate --alerts X --events Y` → compute recall + FP

Code: this repo. Validation artifacts: `gate/`, `events/`, `gate/results/`.